

בדיקות API לבודקת ידנית

פרק A - יסודות HTTP

- **HTTP Request Structure | מבנה בקשת HTTP:** בקשת HTTP מורכבת ממספר חלקים: שורת בקשה (למשל `GET /users?id=1 HTTP/1.1`), כותרות (Headers) ובאדי Body אופציונלי. השורה הראשונה כוללת את המתודה (GET, POST וכו') ואת הנתיב (path) ובפרוטוקול HTTP המתאים. כותרת `Host` למשל מגדירה את השרת אליו נשלחת הבקשה. הגדרת מבנה תקין חיונית - למשל לשלוח כותרת `Content-Type` נכונה כשיש גוף בקשה `[L197-L204+1]`.
- **רמת K: K1** - ידע והבנה.
- **זווית מבחן:** בדיקה שהבנה בסיסית של מבנה הבקשה קיימת: למשל בשאלה על שורות הבקשה והכותרות, או סימולציה בראייה של בקשה ידנית.
- **טעות שכיחה:** לא לציין כותרות קריטיות (למשל `Host` או `Content-Type`) וכך להניח שהשרת יסיר אוטומטית.

- **HTTP Response Structure | מבנה תגובת HTTP:** תגובת HTTP כוללת שורת סטטוס (למשל `HTTP/1.1 200 OK`), כותרות גוף (Headers) ובאדי Response אופציונלי. השורה הראשונה מכילה את קוד הסטטוס שמסקף את תוצאת הבקשה. לדוגמה, קוד 200 אומר שהכל בסדר (OK), 404 אומר משאב לא נמצא. כותרות גוף כמו `Content-Type` מצינות את סוג הפורמט של התגובה.

- **רמת K: K1** - ידע.
- **זווית מבחן:** שאלות על פרשנות השורות הראשונות (מה פירוש קוד סטטוס, מה מכיל ה-Header `Content-Type`).
- **טעות שכיחה:** בדיקה שאינה בודקת גוף תגובה כלל כיוון שהסטטוס היה 200, או להפך - התעלמות מהסטטוס ומבדיקה רק של גוף התגובה.

- **HTTP Methods (GET, POST, PUT, PATCH, DELETE) | מתודות HTTP:** אלה מתודות התקשורת עם השרת. לכל אחת יש משמעות שונה: `GET` מושכת מידע בלבד, `POST` יוצר משאב חדש, `PUT` מחליף משאב קיים במלואו, `PATCH` מעדכן חלק ממשאב, ו-`DELETE` מוחק משאב. חלק מהמתודות אידימפוטנטיות - כלומר קריאה זהה חוזרת לא תשנה את המצב מעבר לקריאה הראשונה. לדוגמה, `GET`, `PUT`, `DELETE` הם בדרך כלל אידימפוטנטיים, בעוד `POST` ו-`PATCH` אינם בהכרח `[L191-L194+28]`.

- **רמת K: K2** - הבנה.
- **זווית מבחן:** היבט השאלה יכול להיות על מושג האידימפוטנטיות או מה יקרה בכפל קריאות (למשל תרחיש שלחת פעמיים `POST` ויצרת שני משאבים במקום אחד).

- **טעות שכיחה:** בלבול בין `PUT` ל-`POST` ובין `PATCH` ל-`POST`; הנחה ש-`POST` הוא אידימפוטנטי או ש-`PATCH` מוחק משאב שלם.

- **Path vs Query Parameters | פרמטרי נתיב מול פרמטרי שאילתה:** ב-URL של API, החלק לאחר שם השרת עד סימן השאלה (?) נקרא נתיב (`path`) ועשוי להכיל מזהים רלוונטיים למשאב; מה שאחר הנקודה-פסיק או סימן השאלה נקרא שורת שאילתה (`query`). למשל `GET /users/5?active=true` - כאן `5` הוא מזהה בתוך הנתיב, ו-`active=true` הוא פרמטר שאילתה. הקוד בשרת מפריד ביניהם: פרמטרי שאילתה מספקים פילטרים או שינויים לבקשה, ואילו הנתיב מגדיר את המבנה ההיררכי של המשאב.

- **רמת K: K1** – ידע.
- **זווית מבחן:** זיהוי נכון של החלקים ב-URL, למשל בשאלה תמונתית על מבנה כתובת ה-API.
- **טעות שכחה:** שימוש שגוי במזהה בנתיב במקום בשורת השאילתה או להיפך. למשל טעות בהתאמת הנתיב כאשר מנסים לקרוא/למחוק משאב לפי מזהה.

פרק B – קודי סטטוס

קודי הסטטוס ב-HTTP מראים האם בקשה הצליחה או נכשלה, ומי האשם. הקודים נחלקים למשפחות: 2xx – הצלחה, 4xx – טעות מצד הלקוח (הבקשה), 5xx – שגיאה מצד השרת.

- **200 OK (הכול תקין):** מציין שהבקשה הצליחה והשרת מחזיר נתונים בבאדי (אם צפוי תוכן). אין אשמה – הכל תקין. בבדיקות נוודא שהתוכן בגוף התגובה תואם את המצופה (מבנה וערכים נכונים).
- **K: K1.**
- **זווית:** שאלה על קוד סטטוס תקין.
- **טעות:** הנחה ש-200 אומר בהכרח שהכל נבדק – יתכן שהשרת השיב קוד 200 אך עם גוף שגוי (Anti-pattern נושא F).
- **201 Created (נוצר בהצלחה):** הבקשה יצרה משאב חדש (למשל POST ששמר אובייקט). השרת מציין זאת ובדרך כלל כולל ב-Location או בתגובה את מזהה המשאב החדש. אין אשמה, רק יש לבדוק שנוצר אובייקט חדש ושמהו חוזר כצפוי.
- **K: K1.**
- **זווית:** מה עושה השרת בקוד זה – נוצר משאב חדש.
- **טעות:** לשכוח לבדוק שהמשאב אכן קיים גם לקריאה ב-GET לאחר מכן.
- **204 No Content (אין תוכן):** הבקשה הצליחה אך השרת לא מחזיר גוף. לדוגמה, בקשת DELETE למחיקה מוצלחת או עדכון בלי צורך בהחזרת גוף. אין אשמה – פשוט אין תוכן לבדוק.
- **K: K1.**
- **זווית:** יש לבדוק שלא מצפה לתוכן תגובה כשיוצא 204, ולהבין שזה תקין שאינו מחזיר גוף.
- **טעות:** לבחון גוף גם כשקוד 204 ואין גוף.
- **400 Bad Request (בקשה שגויה):** הלקוח (הבודק או מערכת קוראת) שלח בקשה במבנה שגוי או עם נתונים לא חוקיים. למעשה, באג בקוח – שולחים נתונים שגויים. בבדיקה נוודא למה השרת לא קיבל את הבקשה (הודעת השגיאה בדרך כלל מפרטת מה שגוי).
- **K: K2** (הבנת אשמה שגיאה בלקוח).
- **זווית:** שאלה על אילו נתונים גרמו לשגיאה – בדיקה של האם הפרמטרים תקינים במבנה הנכון.
- **טעות:** בלבול בין 400 ל-422. 400 הוא על סינטקס כללי, 422 "unprocessable" מורה שלעיתים שגיאת אימות עמוקה יותר.
- **401 Unauthorized (לא מאומת):** הקוד אומר שחסרה אישור תקף. כלומר, חובת אימות לא בוצעה או שקוד ה-Authorization חסר/לא תקין – על הלקוח לספק קרדנציאל מתאים. הבאג הוא בבקשה (למשל שכחת טוקן).
- **K: K1.**
- **זווית:** שאלה על ההבדל עם 403 – כאן הלקוח לא מאומת כלל, בעוד ב-403 הוא מאומת אבל חסר הרשאה.
- **טעות:** סיווג שגוי של טעות זו – למשל לחשוב שזה עניין הרשאה (403) במקום אימות.

• **Forbidden 403 (אסור):** השרת יודע מיהו הלקוח (אימות הצליח) אך הוא חסר הרשאה לבצע את הבקשה. האשם: הגדרות הרשאה או הרשאות משתמש (בדומה ל-401, אך עכשיו הלקוח כבר הוכיח את זהותו).

• **K: K1.**

• **זווית:** הבחנה: לעומת 401, כאן אין בעיה באימות אלא בהרשאות.

• **טעות:** בלבול עם 404 – למשל אם משתמש אינו רשאי לראות משאב מסוים, לעתים מחזירים 404 כדי להסתיר את קיומו. זה גורם לתסבוך בפתרון.

• **Not Found 404 (לא נמצא):** המשאב המבוקש לא נמצא. יכול להיות המזהה שגוי או ה-endpoint לא קיים. כאן האשם בבקשה – מבקשים משהו שאין. הבודק צריך לוודא שהנתיב והפרמטרים נכונים.

• **K: K1.**

• **זווית:** בדיקה מידע שגוי בבקשה או שהמשאב אכן לא קיים.

• **טעות:** שימוש בשגיאה זו במקום 403 (כדי לא לאפשר לקוח לדעת שקיים משאב לחוץ אליו).

• **Conflict 409 (התנגשות):** הבקשה סותרת מצב נוכחי. לדוגמה, 'צירת משתמש חדש עם שם שכבר קיים'. משמש לעיתים במקום 400 כאשר ניתן לתקן את הנתונים. אשמת הלקוח (נתונים שסותרים למדיניות המערכת).

• **K: K2.**

• **זווית:** חיפוש מקרה שבו הנתונים מתקיימים כבר ומשתמשים ב-409. למשל שדה ייחודי כבר קיים.

• **טעות:** טעות הנחה ש-409 הוא בעיה בשרת או רק קריסה; זה לקוח.

• **Unprocessable Entity 422 (נפסל):** הנתונים שנשלחו מבחינים כתקינים תחבירית אך לא עומדים בכללי העסקים (למשל אורך טווח לא תקין). אשמה בבקשה והנתונים שסופקו.

• **K: K2.**

• **זווית:** הבדל מ-400: 422 מציין בעיית לוגיקה או אימות, לא רק מבנה רע.

• **טעות:** להתבלבל ולתייג את הבעיה כ-400 פשוט על אף שהסינטקס תקין (מועדפת שימוש בקוד הטעות המתאים).

• **Internal Server Error 500 (שגיאת שרת):** השרת קרס עקב באג פנימי (למשל לולאה אין-סופית או NullPointerException). האשם בשרת – לא בבקשה. בבדיקות נוודא מה גרם לקריסה על ידי בדיקת לוגים או מצבי קצה.

• **K: K2.**

• **זווית:** אלגוריתם שמנבא שאם מדובר בבעיית שרת, זו לא טעות בקוד הבדיקה אלא צריך פנייה למפתחים.

• **טעות:** לייחס שגיאת 500 לפעולה של הלקוח (למשל לראות ב-500 סתם "משהו לא חוקי בבקשה"). בפועל זו בעיית יישום בשרת.

• **Service Unavailable 503 (השירות לא זמין):** השרת אינו זמין כרגע – עומס יתר או תחזוקה. אשמה בשרת או בתשתית, לא בבקשה. הבודק צריך לוודא זו תקלה זמנית (לעיתים נבדוק בתזמון אחר).

• **K: K1.**

• **זווית:** חשיבה על מקרים של זמינות שרת (לדוגמה, תזמון בדיקות בחוץ שעות עבודה קשות).

• **טעות:** לנסות לבצע בדיקות שוב ושוב ב-503 בלי לבדוק אם באמת השרת זמין (אולי לבדוק עם הצוות או לוגים).

פרק Headers – C ואימות

- **Content-Type (תוכן-סוג):** כותרת HTTP המציינת את הפורמט של גוף הבקשה או התגובה (למשל `application/json` או `text/html`). היא מאפשרת ללקוח או שרת להבין איך לפרש את הנתונים המתקבלים [L197-L204†1]. בבדיקות API צריך לוודא שכשנשלחים או מתקבלים נתונים בבקשה/תגובה הציון נכון – למשל שבתגובה JSON הכותרת היא `application/json`.
- **K: K1.**
- **זווית:** נושאי ניסוח במבחן יכולים להיות זיהוי פורמט התוכן או השלכה על טיפול רגיל.
- **טעות:** השמטת `Content-Type` בבקשה מובילה לכך שהשרת יסבך בפענוח התוכן או יחזיר שגיאה (למשל `Unsupported Media Type 415`).
- **Authorization (הרשאה):** כותרת בקשה הנושאת קרדנציאלים לאימות. בשימוש נפוץ `Authorization: Bearer <token>` להעברת טוקן אבטחה (JWT) שתקף לגישה למשאבים מוגנים [L168-L174†9]. בבדיקות נודא שכל קריאה שמחייבת הרשאה כוללת שורת `Authorization` מתאימה, והטוקן בתוקף.
- **K: K2.**
- **זווית:** בבחינות יתבקשו להסביר את התהליך – למשל השימוש ב-401 והאתגר שתפקיד השרת מחזיר כותרת `WWW-Authenticate`.
- **טעות:** להציב את הטוקן במיקום שגוי (למשל לפרמטר בשורת השאילתה), או ללא המילה `Bearer`.
- **Cookies vs Tokens (עוגיות לעומת טוקנים):** עוגייה (`Cookie`) היא חתיכת נתונים שהשרת שולח לדפדפן, והדפדפן יוסיף אותה אוטומטית לבקשות עתידיות. עוגיות הן **מדינתיות** (השרת שומר session במאגר). **טוקן** (למשל JWT) הוא מחרוזת מקודדת מהשרת שנמצאת אצל הלקוח ומוספים לבקשות תחת הכותרת `Authorization`. טוקנים הם **stateless** – השרת לא צריך לשמור session בצד שלו [L43-L47†11]. בבדיקות API ידנית, משתמשים בטוקנים בבקשות REST ולא בשורות עוגיות (עוגיות רלוונטיות בעיקר בדפדפן).
- **K: K2.**
- **זווית:** הבוחן עשוי לשאול על היתרונות – למשל עוגייה מנוהלת אוטומטית על ידי הדפדפן; טוקן מאפשר אימות מדורג או OAuth.
- **טעות:** לבטוח בכך שהדפדפן יעדכן עוגייה על שרת API אינדיפנדנטי. רבות המערכות שמבקשות טוקן במפורש (Bearer) ולא מסתמכות על עוגיות.

פרק JSON – D ותגובה

- **JSON (JavaScript Object Notation):** פורמט טקסטואלי המבנה מידע כאובייקטים (זוגות מפתח-ערך) ומערכים. ב-JSON כל מפתח חייב להיות במירכאות כפולות, והערך יכול להיות מספר, מחרוזת, בוליאן, null, אובייקט נוסף או מערך [L220-L224†14]. JSON הוא סטנדרט נפוץ לתקשורת API. בבדיקות API נבדוק שתגובת JSON תקינה תחברית וכי טיפוס הנתונים נכונים.
- **K: K1.**
- **זווית:** מבחן עשוי לכלול זיהוי נתונים שחוזרים – אם המחרוזת ללא מירכאות, או מספר בפורמט לא חוקי, זו תקלה סינטקטית.
- **טעות:** להתבלבל בין `null` כערך חוקי לבין השמטת שדה. לדוגמה, אם השדה `phoneNumber` חסר בתגובה, משמעותו שונה מאשר `"phoneNumber": null`. שדה חסר יכול להוות בעיה חיצונית למבדק כי הוא מצביע על שינוי במבנה.
- **Null vs Missing Field (Null מול שדה חסר):** ערך `null` בג'סון מציין שהשדה קיים אך אין בו ערך (המשמעות מוגדרת – למשל לא הוזן). שדה חסר (missing) פירושו שהשדה אינו כלל במבנה התגובה. בבדיקות

יש לשים לב להבדל: למשל תבנית דרישה עלול לדרוש שכל שדה יסופק (אפילו אם null), ואי הימצאותו עשויה להיחשב לפרט חסר.

• **K: K2.**

• **זווית:** נבחנו במבחן מקרים שבהם ציפו שיוחזר שדה עם ערך null, אך הוחזר שדה ריק, או להפך.

• **טעות:** לצפות שהבדיקה תצליח כאשר השדה חסר במקום להיות null, או לפסול JSON תקין רק כי אין ערך (null) הנדרש.

• **Response Validation vs Requirements (אימות תגובה מול דרישה):** בודקת API משווה את התגובה מה-API אל מול מה שמצוין בדרישה או בתיעוד. יש שלוש רמות של אי-התאמה: (1) **סטטוס שגוי** – למשל חזרה 200 במקום הצפוי 201, או להיפך. (2) **מבנה שגוי** – שדות חסרים, רווחים מיותרים או טיפוסים לא תואמים. (3) **ערכי שדות** – שדות שנמצאים אבל ערכם אינו כפי שצריך (לדוגמה, תאריך שגוי). בבדיקה יש לבדוק את שלושת הממדים: קוד הסטטוס, מבנה ה-JSON וכל ערך בהתאם לתנאי הדרישה.

• **K: K3** – יישום.

• **זווית:** תרחישי מבחן עשויים לספק תשובה ודרישה, ולבקש למצוא כל הפרה (בתרגיל עיוני למשל של"ח, או מציאת סטטוס שגוי).

• **טעות:** לא לבדוק סטטוס בכלל משום שהגוף מתאים, או להפך – להסתמך רק על קוד סטטוס טוב בלי לוודא את התוכן.

פרק E – תהליך עבודה

• **Collections & Environments (אוספים וסביבות):** בבודקת ידנית משתמשים בכלים כמו Postman לארגן את בקשות ה-API. אוסף (Collection) הוא קבוצת בקשות קשורות – למשל כל ה-endpoints למשתמשים. סביבה (Environment) מאפשרת להגדיר משתנים גלובליים (כתובת ה-API בסיסית, טוקנים, פרמטרים כלליים) שמתעדכנים בקלות בין סביבות כמו פיתוח ופרודקשן. הכנה נכונה של סביבה מאפשרת לטעון ערכים בקלות בכל בקשה ללא עריכת ה-URL כל פעם.

• **K: K2.**

• **זווית:** הבודק עשוי להידרש להסביר מדוע מפרידים בקשות לאוספים, או תרחיש בו משתמשים במשתנה סביבה (למשל URL בסיסי שמשתנה בין פיתוח לפרוד).

• **טעות:** לפתח כל בקשה בנפרד ללא שימוש באוסף, מה שגורר כפילויות.

• **Chaining Requests (שרשרת בקשות):** תהליך של שימוש באאוטפוט של בקשה אחת כמקור מידע לבקשה הבאה. לדוגמה, ראשית שולחים `POST /users` ליצירת משתמש חדש, מקבלים מזהה, ואז שולחים `GET /users/{id}` כדי לאמת שהמשתמש נוצר כראוי. חשוב לוודא שכל קריאה מממשת (resolve) את הערכים הדרושים לבקשות הבאות ולהגדיר משתנים בממשק הכלי.

• **K: K3.**

• **זווית:** תיאור תרחיש שרשרת – למשל יצירת אובייקט חדש, עדכון ובדיקת המחיקה.

• **טעות:** להריץ `POST` במחזוריות בלי לנקות או לחכות לתגובה, ואז `GET` על מזהה שגוי כי קיבל שם שגוי.

• **Cleanup (ניקוי):** יש להחזיר את הסביבה למצב המקורי לאחר הבדיקות, במיוחד כשנוצרו או שונו נתונים. זה נעשה למשל על ידי קריאת DELETE על הנתונים שנוצרו, או שימוש ב-API ייעודי לכך. שמירת סביבה נקייה חשובה כדי שבדיקות יחזרו תמיד על אותם תנאים ולא ישפיעו זו על זו.

• **K: K2.**

• **זווית:** ערכי הבודק המתודולוגית תחקיר; למשל שאלת הכנה לסביבות – הבודק נדרש להסביר כיצד נוודא שהבדיקה לא תשאיר משאבים (למשל בשינוי בקוד ובבדיקה חוזרת).

- **טעות:** השאירו שימוש במידע שהוחזר בעבר: לדוגמה, בריצה חוזרת של תרחיש מצאו שהבקשה הראשונה פועלת אבל השנייה נכשלת כי המערכת לא מוחקת אחרי שימוש קודם.

פרק F – אנטי-דפוסים בשטח

- **200 OK עם גוף שגיאה:** מתן קוד 200 כאשר בפועל קרתה טעות חמורה הוא אנטי-דפוס. לדוגמה, שרת שחזיר 200 אך תוכן ה-body מציג הודעת שגיאה מפורטת. זה יכול להטעות את המבחן האוטומטי או הידני (יזוהה כהצלחה בטעות). בבדיקה יש לתפוס זאת על ידי עיין בגוף התגובה ובדיקת שדה ה"הצלחה" או "error" בו.
- **K: K2.**
- **זווית:** עלול להיכלל כחלק מסוג תרגיל – למצוא חוסר עקביות קוד-בגוף.
- **טעות:** לבדוק רק קוד 200 ולסמן מייד הצלחה בלי לבדוק תוכן, או להתעלם מבחירת שגיאות שמתחת.
- **הודעות שגיאה חשיפתיות:** שרת שבתגובה לשגיאה מחזיר מידע פנימי (SQL, stack trace, פרטים טכניים) מסכן את האבטחה ופוגם בפרטיות. בבודקת API יש לוודא שההודעה שקיבלה לקוח אינפורמטיבית אבל לא חושפת מידע יתר (לדוגמה "User exists" במקום הסבר פנימי).
- **K: K2.**
- **זווית:** בציון שמתבססים על הנחיות אבטחה, יתכן שיש לשאול על הפרקטיקה המתבקשת (לגרום שגיאות ידידותיות למפתח, לא להוציא מערכות).
- **טעות:** לבדוק שגיאה רק בהתאם לקוד הסטטוס ולדלג על פרטים בגוף – אלה יכולים לעזור במעקב אחר הבאג.
- **אי-עקביות DB↔API↔UI:** במערכות בשכבות, נתון יכול להופיע שונה בממשק המשתמש, בתשובת ה-API ובמאגר הנתונים. אנטי-דפוס שכיח הוא חוסר תיאום בין השכבות (למשל UI מציג שהעדכון הצליח אך ה-API/DB נכשלים). על הבודק למצוא באיזו שכבה הבעיה: אפשר להשוות ערכים ישירות בבסיס, ב-API וב-UI. אם הנתון ב-UI שגוי אבל בבסיס תקין, הבעיה היא בפרונט; אם ב-DB תקין אך ה-API מחזיר ערך אחר, הבעיה בשרת.
- **K: K3.**
- **זווית:** תרחישים של "נתון אחד בשלוש שכבות" שבו יש קונפליקט.
- **טעות:** לנסות לתקן רק על פי מה שה-API מציג בלי לבחון שכבות אחרות – הדבר מוביל לשייך באג שגוי (לדוגמה להאשים frontend כאשר הבעיה היא ב-API).

טעויות נפוצות בבדיקות API

- שימוש בקוד סטטוס 200 כמדד ל"הצלחה" בלי לבדוק את גוף התגובה (התעלמות מתוכן הגוף).
- הבלבול בין 401 Unauthorized ל-403 Forbidden (מחשיבים שגיאה מסוימת במקום הנכון של אימות/הרשאה).
- שגיאה ב-PUT מול PATCH: למשל שימוש ב-PUT כשאין צורך להחליף כלל את המשאב, או פירוק מטענה של פלקסציה.
- לדלג על בדיקת מערך פרמטרים או תלות נכונה בהם (למשל לא לבדוק query ולנחש שהשרת יטפל כשורה).
- השמטת קריאת BODY בבקשה או בתגובה (לדוגמה לא מגדירים Content-Type).
- הרצת תסריטים במסה בלי לשים לב לניקוי תצורה (נתונים חריגים שנערמו מסבכים בדיקות נוספות).
- השערה מוקדמת של הסיבה לבעיה בלי איסוף ראיות (למשל לדלג ישר ל"זו בעיית שרת" במקום לבדוק שאכן נשלחה בקשה נכונה).
- ניסיון בדיקות חוסם (לדוגמה: "Test ID" שהוא בעצם באק-אנד", לא ממש API) במקום לנסח תרחישי קצה נכונים (קלט ריק, JSON לא תקין ועוד).
- חוסר שימוש במשתני סביבה – במקום לשנות כתובת ידנית בכל מעבר בין סביבות, לאנשי בדיקה נוטים לשכוח עידכון.
- נסיון לבדוק POST שוב ושוב כשהשרת מציב מגנוני תשלום (rate limit); הבודק לא משכפל מנגנון נורמלי ואז רואה 429.

מבני תרגיל לדוגמה

להלן מספר דוגמאות לתרגילים של בדיקת התאמת API (בסגנון *api_assertion*). בכל תרגיל ניתנים מפרט בקשה או דרישה, ותגובת API לדוגמה; המשימה היא למצוא את אי-ההתאמות:

• תרגיל 1:

דרישה: יצירת מוצר חדש בעזרת POST צריכה לקבל קוד 201 ותשובת JSON עם שדה "id" מוגדר ב-DB.
תגובה:

```
{
  "status": "success",
  "message": "Product created",
  "id": null
}
```

בדיקה: הקוד היה 200 OK במקום 201, וה- id חוזר כ-null למרות שהמוצר אמור לקבל מזהה. שתי אי-ההתאמות: הסטטוס ושדה id.

• תרגיל 2:

דרישה: פעולה של DELETE /users/5 מוחקת את המשתמש ב-ID=5 ומחזירה 204 No Content.
תגובה:

```
{
  "success": true
}
```

בדיקה: כאן גם קוד הסטטוס חסר (השרת שלח 200 לכל היותר). וגם במקום 204 יש לנו תכולה (body עם השדה success) - שזה סותר את הסיכום שלמחיקה לא צריך תוכן.

• תרגיל 3:

דרישה: GET /inventory?available=true אמור להחזיר מערך של מוצרים זמינים בלבד.
תגובה:

```
[
  { "id": 10, "name": "Chair", "available": true },
  { "id": 12, "name": "Table", "available": false }
]
```

בדיקה: חוסר התאמה במבנה (התוצאה היא מערך במקום אובייקט), ושני הנתונים אינם עומדים בתנאי השאילתה - הפריט השני אינו זמין למרות שהבקשה ביקשה רק זמינים.

• תרגיל 4:

דרישה: PUT /users/2 יעדכן שם ומשימה, ומחזיר את המשתמש המעודכן עם סטטוס 200.
תגובה:

```
{ "id": 2, "username": "alice123", "email": "alice@example.com" }
```

בדיקה: חסרים שדות שצוינו בדרישה (נאמר עדכון שם ומשימה, אך שדה `role` נעלם בתגובה). בנוסף, יתכן שהערך של `email` שונה מהצפוי.

• תרגיל 5:

דרישה: שליחת תיבת טקסט ריקה לשדה חובה ב-POST `/login` תחזיר 422.
תגובה:

```
{ "error": "Password cannot be blank" }
```

בדיקה: החזרה מוצגת כאילו "כל בסדר" (200 OK, הנחה מטעית) עם הודעת שגיאה במקום סטטוס שגיאה מתאים. אי-התאמה: סטטוס לא נכון (היה אמור להיות 422, נפל ב-200).

שאלות בראיונות לבודקת API

- מה ההבדל בין `401 Unauthorized` ל-`403 Forbidden`, ומתי נשתמש בכל אחד?
- הסבר מה עושה המתודה `PATCH` לעומת `PUT` וכיצד זה בא לידי ביטוי בבדיקת API.
- אילו כותרות HTTP חשובות לבדוק ואיך מוודאים שהן נכונות במבחן תרחיש API?
- איך תכתבי קריטריוני קבלה לבדיקת API? (לדוגמה `given-when-then` לבדיקת קריאה ותשובה).
- אילו כלים ידניים בשימוש נפוץ לבדיקת API ואיך מפעילים תרחישי שרשור (`chaining`) במבחן?
- תארי מצב שבו הבדיקה ידנית גילתה באג שהבדיקות האוטומטיות פספסו – מה עשית?
- מה ההבדל בבדיקת `GET` לעומת `POST/Others` בכל הקשור לנתוני אימות?
- בפגישה עם מפתח, הוא מעדכן דרישה של API (לדוגמה משנה שם שדה). איך את מוודאת שהתסריטים קיימים מעודכנים? (מבחן עקיבות).

מונחים נפוצים (עברי/אנגלי)

הנה רשימה של מונחים ומונחים דו-לשוניים שיעזרו בהבנת הבדיקה ובעבודה בשוק הישראלי:

English	עברית (מונח נפוץ)
API – Application Programming Interface	ממשק תכנות אפליקציה (API)
Request	בקשה (HTTP request)
Response	תגובה (HTTP response)
Endpoint	נקודת קצה (API endpoint)
Header	כותרת (HTTP header)
Body	גוף (למשל request/response body)
Status Code	קוד סטטוס
Path	נתיב (ב-URL)
Query Parameter	פרמטר שאילתה (ב-URL)
JSON	ג'ייסון (פורמט תגובה)
Bearer Token	טוקן (Bearer JWT)
Cookie	עוגייה (Cookie)

English	עברית (מונח נפוץ)
Collection	אוסף בקשות (למשל ב-Postman)
Environment	סביבה (variables, משתני סביבה)
Chaining	שרשרת (שליחת מזהה לבקשה הבאה)
Cleanup	ניקוי (של נתונים אחרי בדיקה)
Validation	ולידציה / אימות ערכים
Idempotent	אחיד (idempotent)
Authentication	אימות (למשל Basic/Auth token)
Authorization	הרשאה (גישה)
Response Time	זמן תגובה (latency)
Rate Limit	מגבלת בקשות (אוברייט)

מקורות ופרטי רישיון

טבלת המקורות הבאה מפרטת את מקורות המידע ששימשו ואת פרטי הרישיון של כל מקור:

מקור	URL	מפיץ	שנה	רישיון
– MDN Web Docs Content-Type header 【L197-L204†1】	developer.mozilla.org (MDN)	Mozilla	2025	CC-BY-SA 2.5 (Open)
– MDN Web Docs Authorization header 【L199-L204†3】	developer.mozilla.org (MDN)	Mozilla	2025	CC-BY-SA 2.5 (Open)
MDN Web Docs – JSON 【L220-L224†14】	developer.mozilla.org (MDN)	Mozilla	2025	CC-BY-SA 2.5 (Open)
MDN Web Docs – HTTP- Methods 【27†L194 【L202】	developer.mozilla.org (MDN)	Mozilla	2025	CC-BY-SA 2.5 (Open)
– MDN Web Docs -Idempotent 【28†L188 【L194】	developer.mozilla.org (MDN)	Mozilla	2025	CC-BY-SA 2.5 (Open)
QAJourney: Manual API Testing 【16†L82-L91】	qajourney.net	QAJourney (בלוג)	2026	– QAJourney.net © All rights reserved
ססנת – בדיקות API 【L52-L56†22】	tesnet-group.com	Tesnet Group	2020	© 2020 ססנת – כל הזכויות שמורות
ג'ון ברייס – מדריך Postman 【L104-L112†23】	johnbryce.co.il	ג'ון ברייס	2026	כל הזכויות שמורות (בלוג)

מקור	URL	מפיץ	שנה	רישיון
– Okta Developer Cookies vs Tokens 【L43-L47†11】	developer.okta.com	Okta (בלוג)	2022	Okta – All 2026 © rights reserved
Swagger (Bearer Auth) 【L168-L174†9】	swagger.io (OpenAPI Docs)	SmartBear	2023	All rights reserved (אינו מצוין)